The Comprehensive Landscape of Digital Signal Processing in the Rust Ecosystem with a Strategic Focus on Embedded Systems

The digital signal processing (DSP) domain has historically been a stronghold of C and C++, languages that offered the proximity to hardware and the deterministic performance required for real-time signal manipulation. However, the paradigm is shifting toward the Rust programming language, which provides memory safety, thread safety, and modern expressive power without the runtime overhead typically associated with managed languages. For the embedded systems community, this transition is particularly significant. In environments where a single pointer error can lead to system-wide failure or where race conditions in a multi-core microcontroller can cause catastrophic timing jitter, Rust's compile-time guarantees offer a transformative advantage. This report provides an exhaustive literature review of the current state of Rust DSP crates, categorizing their capabilities, evaluating their suitability for embedded and bare-metal environments, and identifying the critical functional gaps where C and C++ libraries still maintain a superior presence.

I. Architectural Paradigms and the Real-Time Mandate

The architectural philosophy of Rust's DSP ecosystem is bifurcated between the pursuit of generic, high-level abstractions and the necessity of strict, low-level resource management. At the heart of this tension is the requirement for "no_std" compatibility, which ensures that libraries can operate without a standard library, a prerequisite for bare-metal development on microcontrollers like the ARM Cortex-M or RISC-V cores.[1] The state of the art in Rust DSP is defined by how well a crate balances these needs.

The Real-Time Constraint and Memory Management

In a typical DSP application, such as an audio processor or a sensor fusion engine, the signal must be processed within a deterministic time window. Traditional C++ approaches often rely on complex inheritance hierarchies and virtual function calls, which can introduce branch misprediction overhead. Rust's approach, primarily driven by traits and monomorphization, allows for zero-cost abstractions that are resolved at compile time, resulting in inlined code that rivals hand-optimized C.[2]

The community has largely moved toward a modular design, where foundational crates provide the traits for samples and signals, while specialized crates implement specific algorithms. This modularity is a direct response to the fragmentation seen in earlier years, where many audio-related crates existed with overlapping goals and inconsistent APIs.[3] Current efforts, led

by groups such as the Rust Audio organization, aim to unify these into a shared vision, increasing user confidence and ensuring long-term maintenance.[3]

II. Fundamental Sample Abstractions and Fixed-Point Arithmetic

The most basic unit of any DSP system is the sample. In Rust, the dasp suite (Digital Audio Signal Processing) has emerged as the foundational library for working with Pulse-Code Modulation (PCM) data.[4] Its primary goal is to act as a standard library for audio DSP, focusing on portable and fast fundamentals that require no dynamic allocations and have zero external dependencies.[4]

Crate Name	Primary Responsibility	Embedded Utility (no_std)
dasp_sample	Traits and conversions for sample types (i16, f32, etc.)	Essential / High [4]
dasp_frame	Multi-channel frame representations and conversions	Essential / High [4]
dasp_slice	High-performance operations on slices of samples	Moderate / High [4]
dasp_ring_buffer	Fixed-size and bounded buffers for delay lines	Moderate / High [4]
dasp_signal	Iterator-like stream processing API	High / Core [4]

The Fixed-Point Imperative in Embedded Systems

While desktop DSP can rely on the abundance of floating-point units (FPUs), many embedded systems, particularly lower-end microcontrollers like the ARM Cortex-M0 or M3, must perform signal processing using fixed-point arithmetic to maintain performance. The fixed crate is the de facto standard in Rust for representing fixed-point numbers using const generics, allowing for precise control over the number of fractional bits.[5]

The integration of fixed-point arithmetic into DSP algorithms is a significant differentiator between crates. For instance, the idsp crate is specifically designed for embedded use,

providing tuned algorithms for integer-based datatypes.[6] This is critical for applications like high-accuracy Phase-Locked Loops (PLL) and CORDIC-based trigonometric calculations, where floating-point emulation would be too slow.[6]

Algorithm Type	implementation Strategy	accuracy / Performance
cossin()	128-element LUT with linear interpolation	108 dB SNR, 23.5 cycles on Cortex-M7 [6]
atan2()	Complex-to-phase conversion	20.5 bit RMS accuracy, 60 cycles on M7 [6]
Biquad	Fixed-point IIR with anti-windup clamping	i8 to i64 support, dynamically adjustable [6]

The mathematical implications of fixed-point arithmetic in IIR filters are profound. Quantization errors can lead to instability or limit cycles if not handled correctly. Rust's type system allows crates like fixed-filters to enforce the bit-depth of input and output signals at the type level, preventing the silent overflows that plague C-based fixed-point implementations.[8]

III. Modular Audio Signal Processing and Functional Graph Frameworks

Beyond simple sample manipulation, the Rust community has pioneered highly expressive ways to define signal processing chains. FunDSP represents the vanguard of this movement, introducing a "Composable Graph Notation" that allows audio networks to be expressed in algebraic form.[2]

Functional Synthesis and Static Graphs

The core innovation of FunDSP is the use of graph operators to construct signal chains that are compiled into stack-allocated, inlined forms.[2] A notation like sine_hz(f) * f * m + f >> sine() is not merely a shorthand; it represents a structure where the Rust compiler can perform aggressive optimizations, effectively eliminating the overhead of a traditional directed acyclic graph (DAG) representation.

Feature	AudioNode (Static)	AudioUnit (Dynamic)

Allocation	Stack-allocated (no_std friendly)	Heap-allocated (requires alloc)
Arity	Fixed at compile time via types	Determined at runtime
Performance	Maximum (fully inlined)	High (virtual function overhead)
Use Case	Microcontrollers, real-time kernels	Desktop DAWs, modular synthesizers

This dual-system approach allows FunDSP to be used in both highly constrained embedded environments and more flexible desktop applications.[2] Furthermore, its ability to analytically determine frequency responses and causal latencies for linear networks through transfer function composition is a feature typically reserved for offline mathematical software like MATLAB, now available at the library level in Rust.[2]

Core DAW Primitives

While FunDSP focuses on the functional paradigm, audio_core_dsp provides the building blocks for digital audio workstations (DAWs), including oscillators, filters, and envelopes with a clean, trait-based API.[9] It implements PolyBLEP (Polynomial Band-Limited Step) anti-aliasing for its oscillators, which is essential for preventing high-frequency foldover artifacts in sawtooth and square waves, a common issue in simplified embedded synthesis.[9]

The Moog ladder filter implementation in audio_core_dsp uses cached coefficients for efficient per-block processing, demonstrating an understanding of the cache-locality requirements of modern processors.[9] This library is part of a larger ecosystem aimed at providing a complete stack for audio production, including MIDI synchronization and audio graph management.[9]

IV. Scientific Signal Processing and Advanced Spectral Estimation

For tasks extending beyond audio into radar, biomedical engineering, and industrial monitoring, the scirs2-signal crate provides a comprehensive toolkit modeled after Python's SciPy.[10] This

library is part of the SciRS2 ecosystem, which prioritizes a "pure Rust" approach, eschewing dependencies on legacy C or Fortran libraries like BLAS or FFTW in favor of native implementations like OxiBLAS and OxiFFT.[11]

The SciRS2 Ecosystem and Production Readiness

The scirs2-signal crate is described as production-ready, supporting advanced topics such as sparse recovery, time-frequency representations, and system identification.[10] Its inclusion of Kalman filters (EKF, UKF) and matched filtering indicates a shift toward more sophisticated signal processing applications that require high numerical stability.

Module	Algorithms and Capabilities
Filter Design	Butterworth, Chebyshev I/II, Elliptic, Bessel (Analog and Digital) [10]
Spectral Estimation	Yule-Walker, Burg, MUSIC, ESPRIT for super-resolution [10]
State Estimation	Kalman, RTS Smoother, EKF/UKF with square-root formulations [10]
Feature Extraction	MFCCs, Mel filterbanks, Delta coefficients, Pitch (YIN, AMDF) [10]
Denoising	Learnable shrinkage functions, Non-local means for 1-D signals [10]

One of the primary benefits of the scirs2 approach is its "no unwrap" philosophy, which enforces proper error propagation through the Result type.[12] In embedded systems, where an unhandled panic can cause a device to reboot or enter an unsafe state, this rigorous error handling is a significant safety feature compared to the error-prone return-code checking found in C libraries.

Spectral Analysis and MIR

The spectral analysis capabilities of scirs2-signal are particularly robust, offering both parametric and non-parametric estimation methods. The inclusion of MUSIC (MUltiple SIgnal Classification) and ESPRIT (Estimation of Signal Parameters via Rotational Invariance Techniques) allows for frequency estimation with a resolution that exceeds the traditional limits of the Fast Fourier Transform (FFT).[10] Furthermore, for Music Information Retrieval (MIR), the library provides tools for beat tracking, chroma features, and structural segmentation, which are increasingly relevant for edge-computing devices that perform real-time audio analysis.[10]

V. Embedded Hardware Acceleration and Vendor-Specific Bindings

Despite the power of pure Rust, certain DSP tasks require leveraging the specialized hardware accelerators present in many microcontrollers. The Rust community has developed several "sys" crates to provide low-level bindings to vendor-provided DSP libraries.

ARM CMSIS-DSP Integration

The cmsis_dsp library is the gold standard for ARM Cortex-M and Cortex-A processors, offering highly optimized signal processing functions.[13] The Rust bindings, provided by cmsis_dsp and cmsis-dsp-sys-pregenerated, allow Rust developers to tap into this performance.[14]

However, the integration is not without challenges. Some CMSIS functions are defined as inline in the original C headers and are consequently missing from the pre-compiled static libraries, making them inaccessible to the Rust linker.[13] Additionally, many CMSIS functions depend on the C standard library's math functions (e.g., sqrtf), which can cause linker errors in no_std Rust environments unless a provider like libm or micromath is enabled.[13]

Espressif ESP-DSP

For the ESP32 family of chips, Espressif provides esp-dsp, which includes assembly-optimized routines for matrix multiplication, FFTs, and IIR/FIR filtering.[16] While primarily intended for use within the C-based ESP-IDF framework, these can be accessed from Rust through esp-idf-sys or by using the no_std core library approach, though the latter requires more manual setup to expose the hardware's DSP capabilities.[1]

Platform	Optimization Level	Supported Primitives
ARM Cortex-M	SIMD/Hardware FPU	Vector math, FFT, Biquads [14]
ESP32	Assembly-optimized (_ae32, _aes3)	Matrix, Dot product, FFT, Kalman [16]
RISC-V	Experimental P-ext / V-ext	Basic arithmetic, emerging vector support [18]

The current state of RISC-V DSP in Rust is more experimental. While the architecture supports "P" (Packed-SIMD) and "V" (Vector) extensions designed for signal processing, high-level Rust abstractions for these instructions are still in development.[18] Most current RISC-V Rust development focuses on the hypervisor extension and core register access rather than

specialized DSP acceleration.[21]

VI. Software Defined Radio and the Communication Protocol Gap

Software Defined Radio (SDR) represents the pinnacle of DSP complexity, requiring high-throughput data processing and sophisticated synchronization. In this domain, the Rust ecosystem is dominated by FutureSDR, a framework designed for building real-time DSP pipelines.[22]

FutureSDR vs. Legacy Frameworks

FutureSDR aims to provide the flexibility of GNU Radio while leveraging Rust's concurrency model to support heterogeneous computing (e.g., CPU, FPGA, GPU).[22] While it is highly capable as a framework, the "blocks" available within the ecosystem are less comprehensive than those found in the C-based liquid-dsp library.[23]

The liquid-dsp library is a critical benchmark for embedded SDR. Written entirely in C to minimize overhead, it provides an exhaustive suite of filters, modems, and synchronizers without relying on external dependencies.[23] For Rust to reach parity, it must develop more robust implementations of the following:

- **Complex Modems**: While basic AM/FM is present, advanced modems like Continuous Phase Frequency-Shift Keying (CPFSK) and Gaussian Minimum-Shift Keying (GMSK) are rare in the Rust ecosystem.[25]
- **Synchronization Blocks**: The critical "meat" of a radio receiver—timing recovery, carrier recovery, and Costas loops—is largely missing in a unified form.[25]
- **Error Correction (FEC)**: Standardized FEC implementations for Viterbi decoding or Reed-Solomon are fragmented across multiple small crates rather than being integrated into a major DSP library.[25]

VII. Comparative Analysis: Identifying the Functional Voids Relative to C and C++

To understand what currently exists in C and C++ but is missing in Rust, one must look at mature libraries like KFR, SigLib, and Numerix SigLib. These libraries represent decades of optimization and feature expansion.

Advanced Filter Design and Analysis

The pm-remez crate provides a modern implementation of the Parks-McClellan Remez exchange algorithm for FIR filter design.[26] While this is a significant step forward, C++ libraries like KFR offer a much wider range of filter design prototypes and transformation functions.[27]

Feature	C/C++ Status (e.g., KFR, SigLib)	Rust Status
Filter Design	Exhaustive (Butterworth to Elliptic)	Robust in scirs2, pm-remez [10]
Vectorization	Highly optimized across all SIMD	Growing (std::simd), platform-specific
Resampling	Arbitrary irrational rate, polyphase	Good in dasp, scirs2 [4]
Comm. Primitives	Fully matured (Modems, Sync)	Limited / Fragmented [25]
Diagnostic Tools	Built-in plotting and frequency charts	Developing (e.g., FunDSP ASCII charts) [2]

The "Standard Library" Gap

One of the most significant absences in Rust is a single, "batteries-included" DSP library that is comparable to MATLAB's Signal Processing Toolbox. While scirs2-signal is making rapid progress in this direction, it is not yet as universally adopted as SciPy is in the Python world.[10] The fragmentation of the audio ecosystem, though improving, still leads to a "choose your own adventure" experience for new developers, whereas a C++ developer might simply reach for a monolithic library like SigLib or the IPPS (Integrated Performance Primitives) from Intel.[3]

Optimization for Modern Instruction Sets

While Rust has good support for ARM and x86 SIMD, it lacks the exhaustive, hand-tuned assembly libraries for niche architectures that have been part of the C ecosystem for years. For example, the RISC-V "P" extension and "V" extension support in Rust is still largely experimental at the compiler level, and high-level DSP crates that can auto-vectorize for these targets are still on the horizon.[19]

VIII. Conclusion: The Roadmap Toward an Integrated DSP Standard

The state of DSP in Rust is one of rapid maturation and structural innovation. The community has successfully moved past the "toy" phase, producing production-ready crates like dasp, FunDSP, and scirs2-signal that offer performance comparable to C++ while providing superior

memory safety.[4] For embedded systems, the combination of no_std support and high-performance fixed-point arithmetic via idsp and fixed makes Rust a viable, and often preferable, choice for new projects.[5]

However, to fully displace C and C++ in established industries, the Rust community must address several key areas:

1. **Unified Communication Stacks**: There is a dire need for a pure-Rust library that replicates the breadth of liquid-dsp, providing standardized modem blocks and synchronization primitives for SDR.[25]
2. **Architecture-Specific Acceleration**: Developers need easier, safer access to the DSP instructions of the ESP32 and RISC-V platforms without resorting to complex FFI and linker hacks.[16]
3. **Standardization of Traits**: The ongoing work by the Rust Audio organization to define a "de-facto tech stack" must continue to prevent further fragmentation and ensure that different DSP crates can interoperate seamlessly.[3]

The mathematical foundations of DSP—IIR filters, FFTs, and spectral estimation—are now well-represented in Rust. The remaining challenges are largely a matter of library breadth and ecosystem consolidation. As more developers migrate from C++ to Rust, drawn by the promise of memory safety in high-stakes embedded environments, it is inevitable that the remaining functional gaps will be closed. The future of embedded DSP is increasingly written in Rust, and the current state of the ecosystem, while still in flux, provides a solid foundation for the next generation of high-reliability signal processing systems.

The mathematical model for IIR filters provides a clear example of the implementation precision required:

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] + a_1y[n-1] + a_2y[n-2]$$

Rust crates like biquad and idsp implement this using numerically stable forms like Direct Form 1 or Direct Form 2 Transposed, ensuring that the theoretical stability of the filter is preserved during execution on constrained hardware.[7] This level of care, combined with the language's inherent safety, defines the current high bar for the Rust DSP community.

Works cited

1. Embedded sensor application with Rust on ESP32 - TiMaMi Software, accessed May 2, 2026, https://timami.de/embedded-sensor-application-with-rust-on-esp32/
2. SamiPerttu/fundsp: Library for audio processing and ... - GitHub, accessed May 2, 2026, https://github.com/SamiPerttu/fundsp
3. RustAudio/audio-ecosystem: An effort to try and unify audio crates - GitHub, accessed May 2, 2026, https://github.com/RustAudio/audio-ecosystem

4. RustAudio/dasp: The fundamentals for Digital Audio Signal ... - GitHub, accessed May 2, 2026, https://github.com/rustaudio/dasp
5. fixed - crates.io: Rust Package Registry, accessed May 2, 2026, https://crates.io/crates/fixed
6. GitHub - quartiq/idsp: Rust DSP algorithms. Often integer math. no-std., accessed May 2, 2026, https://github.com/quartiq/idsp
7. Biquad - QUARTIQ, accessed May 2, 2026, https://quartiq.de/stabilizer/firmware/idsp/iir/struct.Biquad.html?search=Option%3CT%3E,+(T+-%3E+U)+-%3E+Option%3CU%3E
8. fixed-filters - crates.io: Rust Package Registry, accessed May 2, 2026, https://crates.io/crates/fixed-filters
9. audio_core_dsp - crates.io: Rust Package Registry, accessed May 2, 2026, https://crates.io/crates/audio_core_dsp
10. scirs2-signal - crates.io: Rust Package Registry, accessed May 2, 2026, https://crates.io/crates/scirs2-signal
11. scirs2 · PyPI, accessed May 2, 2026, https://pypi.org/project/scirs2/
12. scirs2 - crates.io: Rust Package Registry, accessed May 2, 2026, https://crates.io/crates/scirs2
13. samcrow/cmsis_dsp.rs: Rust bindings to the CMSIS-DSP library for Cortex-M processors - GitHub, accessed May 2, 2026, https://github.com/samcrow/cmsis_dsp.rs
14. cmsis_dsp - Rust - Docs.rs, accessed May 2, 2026, https://docs.rs/cmsis_dsp
15. cmsis_dsp_sys_pregenerated - Rust - Docs.rs, accessed May 2, 2026, https://docs.rs/cmsis_dsp_sys_pregenerated
16. espressif/esp-dsp: DSP library for ESP-IDF - GitHub, accessed May 2, 2026, https://github.com/espressif/esp-dsp
17. ESP-DSP Library User Guide - Espressif Documentation, accessed May 2, 2026, https://documentation.espressif.com/esp-dsp/en/latest/esp32/esp-dsp-library.html
18. RISC-V Vector Processing: Enhanced by Custom DSP Instructions - Synopsys, accessed May 2, 2026, https://www.synopsys.com/articles/signal-processing-risc-v-dsp-extensions.html
19. D108189 [RISCV] Support experimental 'P' extension 0.9.11 - LLVM Phabricator archive, accessed May 2, 2026, https://reviews.llvm.org/D108189
20. Bringing up RISC-V Interrupts using Rust Programming Language - Trepo, accessed May 2, 2026, https://trepo.tuni.fi/bitstream/10024/147785/2/AhmedAisha.pdf
21. riscv_h - Rust - Docs.rs, accessed May 2, 2026, https://docs.rs/riscv-h
22. futuredsp - Rust - Docs.rs, accessed May 2, 2026, https://docs.rs/futuredsp
23. jgaeddert/liquid-dsp: digital signal processing library for software-defined radios - GitHub, accessed May 2, 2026, https://github.com/jgaeddert/liquid-dsp
24. liquid-dsp-1.2.0.pdf, accessed May 2, 2026, https://www.liquidsdr.org/downloads/liquid-dsp-1.2.0.pdf
25. Documentation - liquid-dsp, accessed May 2, 2026, https://liquidsdr.org/doc/
26. maia-sdr/pm-remez: Parks-McClellan Remez FIR design algorithm - GitHub,

accessed May 2, 2026, https://github.com/maia-sdr/pm-remez
27. KFR | Fast, modern C++ DSP framework, accessed May 2, 2026, https://www.kfrlib.com/
28. pm_remez - Rust - Docs.rs, accessed May 2, 2026, https://docs.rs/pm-remez
29. SigLib Digital Signal Processing (DSP) Function List, accessed May 2, 2026, https://numerix-dsp.com/siglib/functions.html
30. biquad - crates.io: Rust Package Registry, accessed May 2, 2026, https://crates.io/crates/biquad